

UNIVERSIDAD AUTONOMA DE TAMAULIPAS.

UNIDAD ACADEMICA MULTIDISCIPLINARIA DEL MANTE.

PROGRAMACIÓN DE INTERFAZ DE PUERTOS.

LABORATORIO 06 – 09



Nombre: Hernández Castro Abigail.

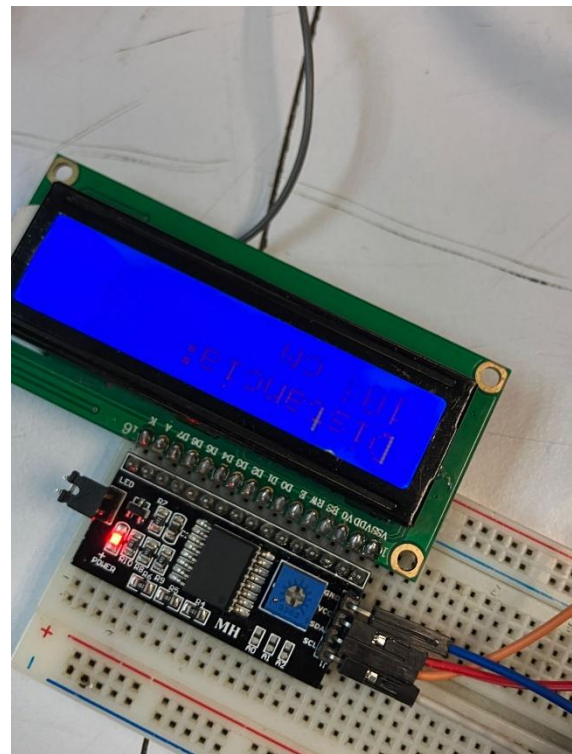
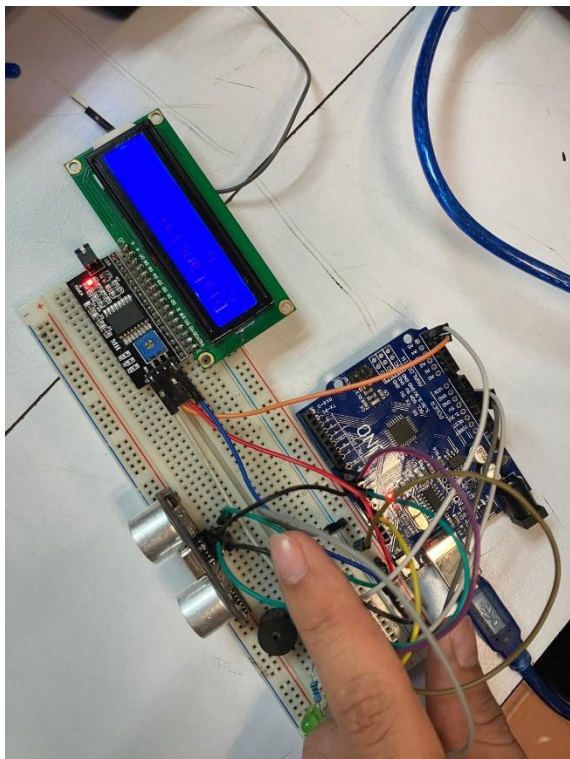
Docente: López Piña Daniel.

15 de abril del 2026.

LABORATORIO 06 SENSOR DE ULTRASONIDO HC-SR04 CON DISPLAY

Objetivo:

Aprenderemos a implementar un sistema de medición de distancia utilizando el sensor ultrasónico HC SR04, dominando el uso de pulsos digitales de tiempo, la función `pulseIn()` y la representación de datos en un display LCD I2C.



Materiales:

- Arduino UNO
- Módulo HC-SR04
- LCD 16x2 con adaptador I2C
- LED
- Buzzer activo
- 12 cables Dupont

CÓDIGO:

// LAB 6: Sensor HC-SR04 con LCD I2C

// Instalar librerías: Wire.h (incluida) + LiquidCrystal_I2C

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
const int TRIG = 9;
```

```
const int ECHO = 10;
```

```
const int LED = 13;
```

```
const int BUZZ = 8;
```

```
const int ALERTA_CM = 20; // Distancia de alerta
```

```
long medirDistancia() {
```

```
digitalWrite(TRIG, LOW); delayMicroseconds(2);
```

```
digitalWrite(TRIG, HIGH); delayMicroseconds(10);
```

```
digitalWrite(TRIG, LOW);
```

```
long dur = pulseIn(ECHO, HIGH, 30000); // Timeout 30ms
```

```
return dur == 0 ? -1 : (dur * 0.0343 / 2);
```

```
}
```

```
void setup() {
```

```
pinMode(TRIG, OUTPUT);
```

```
pinMode(ECHO, INPUT);
```

```
pinMode(LED, OUTPUT);
```

```
pinMode(BUZZ, OUTPUT);
```



**Unidad Académica
Multidisciplinaria
Mante**

Serial.begin(9600);

```
lcd.init();

lcd.backlight();

lcd.print("HC-SR04 Listo!");

delay(1500);

}

void loop() {

long dist = medirDistancia();

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("Distancia:");

lcd.setCursor(0, 1);

if (dist < 0) {

lcd.print("Sin objeto");

digitalWrite(LED, LOW);

digitalWrite(BUZZ, LOW);

} else {

lcd.print(dist); lcd.print(" cm");

Serial.println(dist);

bool alerta = (dist < ALERTA_CM);

digitalWrite(LED, alerta);

digitalWrite(BUZZ, alerta);

}

delay(300);

}
```

Evaluación

- **¿Por qué se divide entre 2 la fórmula de distancia?**

Porque el sonido va ida y vuelta: sale del sensor, rebota en el objeto y regresa. Entonces el tiempo que mide es de todo el recorrido, por eso se divide entre 2 para obtener solo la distancia de ida.

- **¿Qué significa el timeout en pulseIn()? ¿Qué pasa si se omite?**

El timeout es el tiempo máximo que Arduino espera a que llegue el pulso.

Si no lo pones, el programa se puede quedar trabado esperando si no recibe señal.

- **Modifica el código para mostrar un 'gráfico de barras' en el LCD proporcional a la distancia.**

- **Investiga los pines SDA/SCL de Arduino y explica el protocolo I2C con sus condiciones de inicio y parada.**

Los pines SDA y SCL en Arduino sirven para que los dispositivos se comuniquen. SDA envía la información y SCL marca el ritmo. Un dispositivo controla la comunicación y los demás responden. La comunicación empieza cuando la señal baja y termina cuando vuelve a subir.

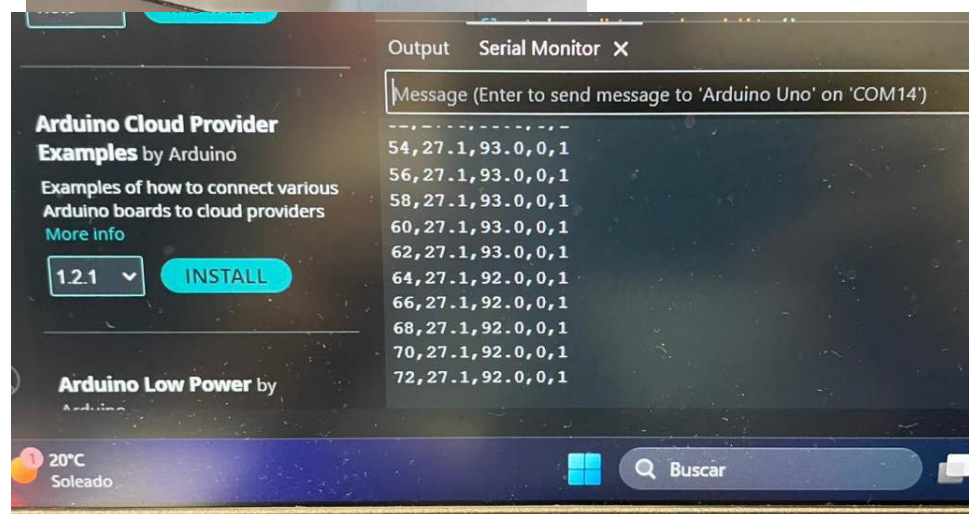
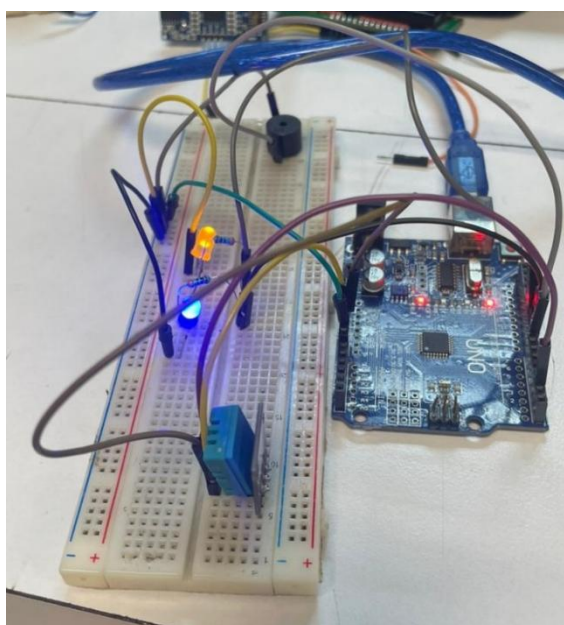
Conclusión:

En esta práctica se logró medir distancias usando el sensor ultrasónico HC-SR04 y mostrar los resultados en un display LCD. Se entendió cómo funciona el envío y recepción de señales, así como el uso de pulseIn() para calcular el tiempo. Además, se aprendió a visualizar la información de forma clara, lo que ayuda a comprender mejor los datos. En general, fue una actividad útil para aplicar conceptos básicos de sensores y comunicación en Arduino.

LABORATORIO 7 SENSOR DE TEMPERATURA Y HUMEDAD DHT11 CON ALARMA CLIMÁTICA

Objetivo:

Aprenderemos a leer la temperatura y humedad relativa con el DHT11, comprenderá el protocolo de comunicación 1-Wire del sensor, implementará lógica de umbrales con histéresis y transmitirá datos formateados vía Serial para visualización.





**Unidad Académica
Multidisciplinaria
Mante**

CODIGO:

```
// Librería necesaria: DHT sensor library by Adafruit

#include <DHT.h>

#define DHT_PIN 2

#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

const float TEMP_ON = 25.0, TEMP_OFF = 26.0;

const float HUM_ON = 80.0, HUM_OFF = 75.0;

const int LED_TEMP = 9, LED_HUM = 10;

const int BUZZER = 8;


bool alertaTemp = false, alertaHum = false;

void setup() {
  dht.begin();

  pinMode(LED_TEMP, OUTPUT);
  pinMode(LED_HUM, OUTPUT);
  pinMode(BUZZER, OUTPUT);

  Serial.begin(9600);

  Serial.println("Tiempo(s),Temp(C),Hum(%),A_Temp,A_Hum");
}

void loop() {

  float t = dht.readTemperature();

  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {

    Serial.println("Error: sensor no conectado");
```

```
delay(2000); return;
}
// Lógica con histéresis
if (!alertaTemp && t >= TEMP_ON) alertaTemp = true;
if ( alertaTemp && t <= TEMP_OFF) alertaTemp = false;
if (!alertaHum && h >= HUM_ON) alertaHum = true;
if ( alertaHum && h <= HUM_OFF) alertaHum = false;
digitalWrite(LED_TEMP, alertaTemp);
digitalWrite(LED_HUM, alertaHum);
digitalWrite(BUZZER, alertaTemp || alertaHum);
Serial.print(millis()/1000); Serial.print(",");
Serial.print(t,1); Serial.print(",");
Serial.print(h,1); Serial.print(",");
Serial.print(alertaTemp); Serial.print(",");
Serial.println(alertaHum);
delay(2000); // DHT11 necesita >= 1s entre lecturas
}
```

Evaluación

- **¿Qué ventaja tiene la histéresis frente a un umbral simple?**

Evita que el sistema esté encendiéndose y apagándose todo el tiempo cuando el valor está cerca del límite. Hace que todo sea más estable.

- **¿Por qué el DHT11 necesita al menos 1 segundo entre lecturas?**

Porque el sensor es lento y necesita tiempo para tomar una nueva medición. Si lees muy rápido, te puede dar datos incorrectos.

- **Compara DHT11 vs DHT22. ¿En qué proyectos conviene cada uno?**

DHT11: más barato y sencillo, sirve para proyectos básicos.

DHT22: más preciso y mide mejor, ideal para proyectos más serios.

Conclusión:

En esta práctica se comprendió la importancia de obtener mediciones estables y confiables, así como respetar los tiempos de lectura de los sensores. También se aprendió a elegir el sensor adecuado según la precisión que se necesite. En general, fue útil para entender cómo mejorar el funcionamiento de sistemas con temperatura y humedad.